



STAY TUNED. This LIVE program will begin shortly.

In the meantime, feel free to visit our social media pages for IT Society, as well as events like Tech Career Days, CodeNecton and BarCamp.



itsocietymmu



techcareerdays_mmu



code_necton



barcamp_cyberjaya



Git and GitHub Workshop



Git

1. Git is installed and maintained on your local system (rather than in the cloud)
1. Git is a version control system that intelligently tracks changes in files.

GitHub

1. GitHub is a cloud-based platform
1. You can store, share, and work together with others to write code.



Using the Command Line

Windows:

Whenever you see "run this command" or "run it" means you should:

1. Open the program "**Command Prompt**" if you don't already have it open.
2. Type (or paste) the command, and press the **Enter** key.
3. E.g. Run the command `cls`

```
C:\Users>cls
```

Mac:

Whenever you see "run this command" or "run it" means you should:

1. Open the program "**Terminal**" if you don't already have it open.
2. Type (or paste) the command given, and press the **Enter** key.
3. E.g. Run the command `clear`

```
al@theendoftheinternet:~$ % clear
```



Command Line – Navigating Folders

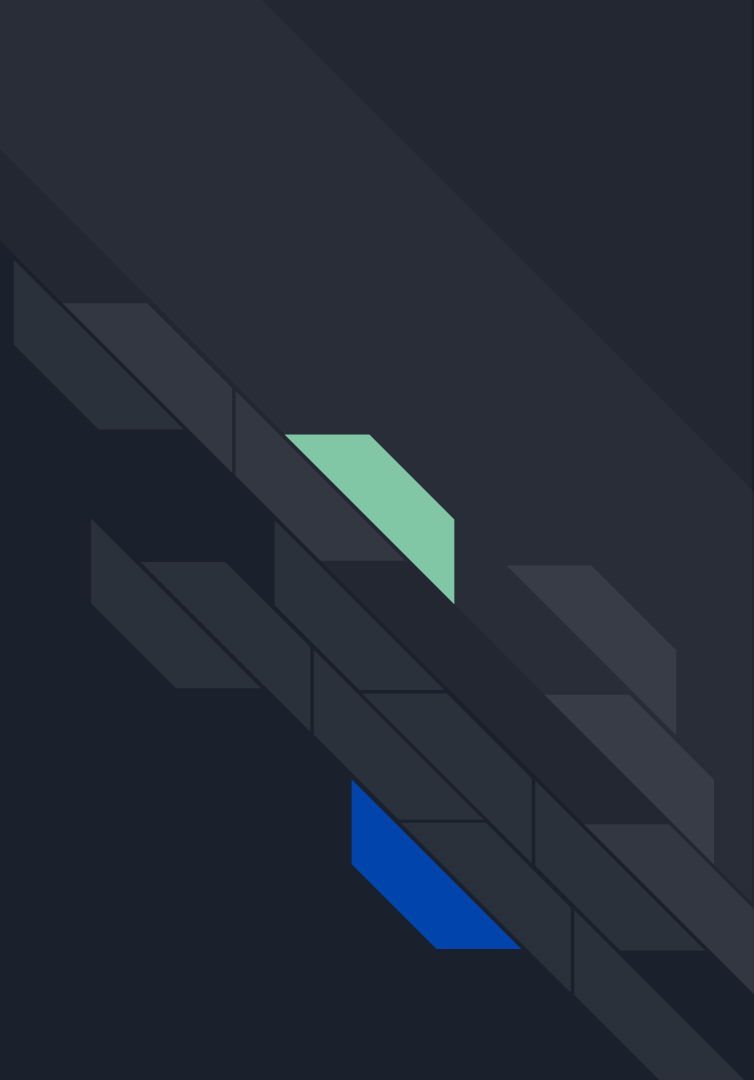
Windows:

1. Your prompt always starts in your user folder. It shows `C:\Users\AccountName>`
2. Try running these commands:
`dir` – see what's in the current folder
`cd foldername` – go into foldername
`cd ..` – go up/out of current folder
`cls` – clear screen
3. You need at least these commands to 'move around' your computer and be at the right folder in your computer to start your repository.
4. You can type a file/folder name halfway and press the "Tab" key to autofill it.

Mac:

1. Your prompt always starts in your user folder. It (usually) shows `username@Macname ~ %`
2. Try running these commands:
`ls` – see what's in the current folder
`cd foldername` – go into foldername
`cd ..` – go up/out of current folder
`clear` – clear screen
3. You need at least these commands to 'move around' your computer and be at the right folder in your computer to start your repository.
4. You can type a file/folder name halfway and press the "Tab" key to autofill it (case sensitive!)

1. Using Git (Solo Mode)





Let's Start a Project

1. What's a project in coding? It's a bunch of text files containing your source code.
2. For this exercise, we're going to write a 'short line by line story' instead of actual coding so we can focus on git.
3. Create a New Folder somewhere on your computer. Git tracks a whole folder at a time, so you **MUST** start an empty new folder. I suggest:
`\Users\YourAccount\Projects\GitExercise\` (same for Mac/Win)
4. Make this folder using Windows Explorer / Finder (Mac) so that it's easier for you.
5. Inside this folder, create a new text file and name it `story.txt`.
Windows - you can just right-click empty folder and select New, Text Document.
Mac - use Visual Studio Code to make the file in the correct folder.
6. Then, go to the command prompt and use the `dir` or `ls` and `cd` commands to go to the folder you created earlier in your command line ([refer to Slide 4](#)). You should see **ONLY** `story.txt` in both your GUI and in your command line. Try running `dir` or `ls` in the command line to confirm that you're in the correct folder. If you see ALL your files, you're in the wrong location.



Filling story.txt

1. Fill `story.txt` with this story – use whatever text editor:

```
Adam wakes up in the morning.  
He brushes his teeth.  
He has breakfast.  
Then he . . .
```

1. Save the text file.



Starting a Git Repository – `git init`

1. We now have a project folder with one text file inside.
2. Tell git to start tracking this project. Make sure your command prompt is IN the same folder as `story.txt`. (You did this two slides ago.)
3. Run the command `git init`



Behind the scenes – what did `git init` do?

Windows

1. Run `dir /a` (with cmd as admin)
2. Run `dir`
3. Compare the two and you should see that the first one shows an extra hidden `.git` directory inside your folder.
 - Don't mess with this folder. Git automatically stores all the messy information needed to keep track of everything you do to this project. Whenever you run a git command, git is working with information in this folder.

Mac

1. Run `ls -a`
2. Run `ls`
3. Compare the two and you should see that the first one shows an extra hidden `.git` directory inside your folder.
 - Don't mess with this folder. Git automatically stores all the messy information needed to keep track of everything you do to this project. Whenever you run a git command, git is working with information in this folder.



Seeing your repo's status – `git status`

- Run `git status`
- You should see `story.txt` in red. It's NOT TRACKED and NOT STAGED.
- This is a useful command to see if you have any files you haven't tracked properly. (Don't do anything yet, just for your info/learning.)
- Use this command often.



Adding files – `git add`

1. So far we have a project folder with one text file inside, and we started a git repo that haven't started tracking anything.
2. Run `git add .`

Note: The `add` command usually works with a specific file name (e.g. `git add somefile.cpp`). When we're lazy, we tell it to add 'the whole current directory' which is what the `"."` or period means.



Seeing your repo's status – `git status`

- Run `git status`
- You should see `story.txt` in green. It's TRACKED and STAGED but NOT COMMITTED.
- Some terms:
 - a. **Track** - The files with changes which Git is paying attention to. (You can tell git to never track certain files by [adding them to a .gitignore](#) file. If you already have files you accidentally committed, [remove them using a command](#).)
 - b. **Stage** - The tracked files with changes which Git will commit when you run the commit command.
 - c. **Commit** - "Saves" the all the tracked files into the repository. You can return your project to any commit you have made previously if you want to (like Google Docs version histories)



Commit your files – `git commit`

1. Run `git commit -m "Initialise repository."`
2. This is your first commit.
3. Run `git status` again and you'll see that your repository is clean - i.e. no new changes.

Note: After the first commit, you should write your own commit messages after `-m` explaining what you added/changed in this version. The double quotes is a must! A good habit is to prefix your commit messages:

`fix: broken link in page 3 fixed` or

`feat: added button for clear all` or

`chore: removed unused files` (feat stands for adding a new feature).



Another commit

1. Let's keep going. Add a new line at the bottom of `story.txt` with the line `He goes to class.` (Ignore the line with `'...'` first. **REMEMBER TO SAVE THE TEXT FILE.**)
2. Run `git status` again. Git knows there's changes. So we now need to `add` before we `commit` these changes. You should know how to do this based on what you've learned in the previous few slides.



See what changed – `git log` / `git show`

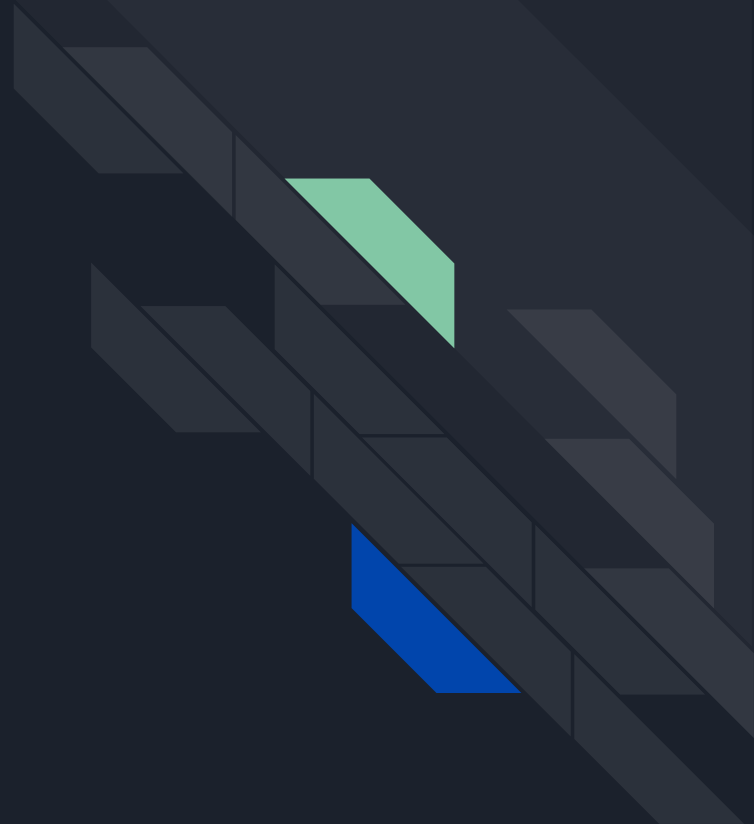
1. Run `git show`. This is the details of your latest commit. You can see which lines changed.
2. Run `git log`. This lists out a bunch of recent commits. (If there is more than one line, you can scroll up and down with the arrow keys and have to press 'q' to quit the view)
3. We seldom use these commands since viewing these things in a GUI like GitHub is usually prettier. But you may want to view something quickly once in awhile.



Renaming a branch – `git branch`

1. You may not realise, but you're right now working in a default branch called `master`. We're going to rename this to `main` based on modern conventions.
2. Run `git branch`. You can see the existing branch name.
3. Run `git branch -m main`
4. Run `git branch` again. You can see the result of the rename.

Branches





Discussion – Why Work in Branches?

- You should always start a new branch when working on a new feature. Even when fixing a bug.
But WHY?
- Situation 1 – "Ok I give up, I can't get this feature to work. Let's just build something else."
It lets you **'undo' all your changes** with one magical command.
- Situation 2 – "Bro! The code you committed that we all pulled already – got error lah!
Now it can't run." It lets you **'pause' your new feature, and go back to main to fix a bug for others**. Your fixes can be sent out to everyone else AND merged into your current feature branch.
- There are many other situations where it's good to isolate your code and keep bookmarks of where you are in your development.



Creating branches

We want to tell the story of what happens at class now. But just in case, we're going to branch out and make our changes in a separate branch from what we have committed so far.

There are 2 ways to create branches, one is more conventional, and the other is quicker and easier. We'll learn the right way to do it first, as to understand how branches are created step-by-step.

Method 1: `git checkout` / `git branch`

`git branch new-branch`

This creates a branch named “new-branch”
(replace with your own branch name)

`git branch`

This shows the list of branches
(We use it here to check if the branch has been created)

`git checkout new-branch`

or

`git switch new-branch`

Both of these commands will switch
to the new branch.

`git push origin new-branch`

Push the branch to GitHub
(this will be covered later in Working as a Team)

Start doing stuff!

Method 2: **git checkout -b**

`git checkout -b new-branch`

This creates a branch named “new-branch”
(replace with your own branch name)

`git branch`

This shows the list of branches
(We use it here to check if the branch has been created)

`git push origin new-branch`

Push the branch to GitHub
(this will be covered later in Working as a Team)

Start doing stuff!

- ❖ note that you don't have to switch into the branch once you created it, since 'checkout -b' creates the branch, then switches to it.



Method 2: `git checkout -b`

1. Run `git checkout -b at-the-class` (Use `-b` only when making a new branch.)
2. Run `git branch`. You can see that there are two 'branches', one of which was `main` – your original branch.
3. In `story.txt`, add new lines at the bottom for things that happened at class. **SAVE** the file.
4. Commit these changes to your new branch. Do the two steps from earlier: `add` and `commit`
5. Check with `git status` to make sure that you're on branch `at-the-class` with a clean directory.

Branch Check



- Right now you have two branches.
- `at-the-class` have a few more lines at the bottom than `main`



Branching – git checkout (cont...)

1. Now, let's go back to the main branch – we forgot to add something earlier.
2. Run `git checkout main`
3. Notice what happens to your `story.txt` file that's open in Visual Studio Code. Your new lines at the bottom of the file have disappeared – they don't exist in the main branch.
4. Let's edit our story and add this new line. Then **SAVE** the file. You do bath in the morning, right?

```
1  Adam wakes up in the morning.  
2  He brushes his teeth.  
3  He takes a bath. |  
4  He has breakfast.  
5  Then he . . .
```



5. Add and commit your changes. These are committed to main since you're at this branch currently.

Branch Check



- Right now you have two branches.
- `at-the-class` have a few more lines at the bottom.
- `main` have an additional line at line 3 about bathing.

Merging committed changes – `git merge`

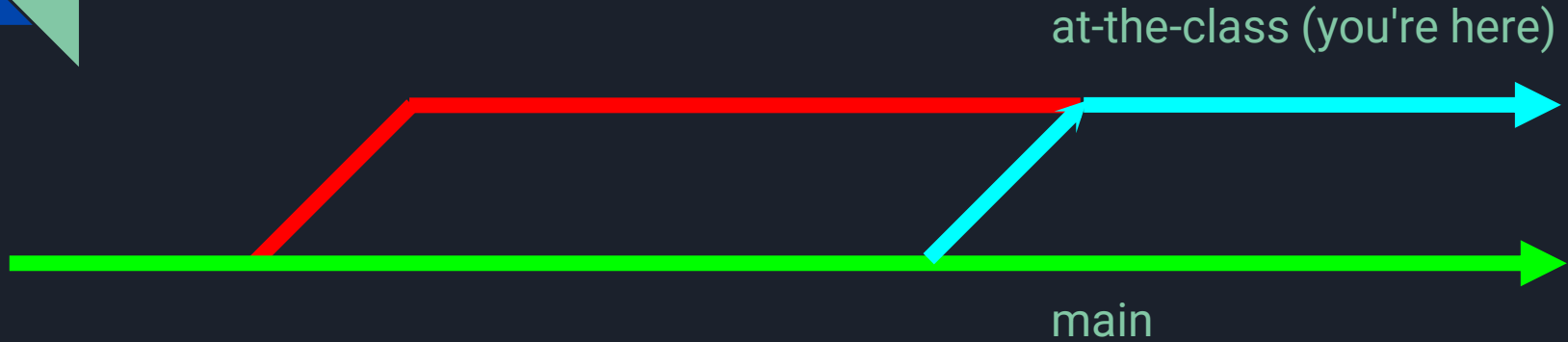
1. We now need to do what we did in the `main` branch, back in our `at-the-class` branch.
Go back to the `at-the-class` branch.
2. Run `git checkout at-the-class`
3. Run `git merge main`. This should open a text file in Visual Studio Code. You can just close this text file.
(It gives an automatic commit message).



The screenshot shows the Visual Studio Code interface. The top bar displays two open files: `story.txt` and `MERGE_MSG`. A red arrow points to the `MERGE_MSG` tab with the text "Just close it". The `MERGE_MSG` file is open in the editor, showing the following content:

```
Users > althras > Projects > GitExercise > .git > MERGE_MSG
1 Merge branch 'main' into at-the-class
2 # Please enter a commit message to explain why this merge is necessary,
3 # especially if it merges an updated upstream into a topic branch.
4 #
5 # Lines starting with '#' will be ignored, and an empty message aborts
6 # the commit.
7
```

Branch Check



- Right now you have two branches.
- **at-the-class** have a few more lines at the bottom.
- **main**'s additional line about bathing have been merged into the branch.



Merging with conflicts – `git merge`

1. Now let's try something different. Edit `story.txt` and change `Then he . . .` to `Then he checked Instagram`. Save file, `git add` and commit this to the current branch (`at-the-class`).
2. Go back to the `main` branch, and edit the same line, but make it say `Then he watched YouTube` instead. (Don't forget to save, add and commit!)
3. Just like we merged changes in `main` into a branch, we can merge a branch into `main` as well.
4. Now that you're in the `main` branch, run `git merge at-the-class`.
5. You will see a merge conflict. Previously this did not occur because git can tell when a new line is added where there wasn't any. Now, git can't tell whether to keep Instagram or YouTube.
6. Git will open the file with the conflict (in this case it's `story.txt`), mark the places with the conflict, and ask you to edit the file until the file is what you want, before committing it. (See next slide)

Merging with conflicts – git merge

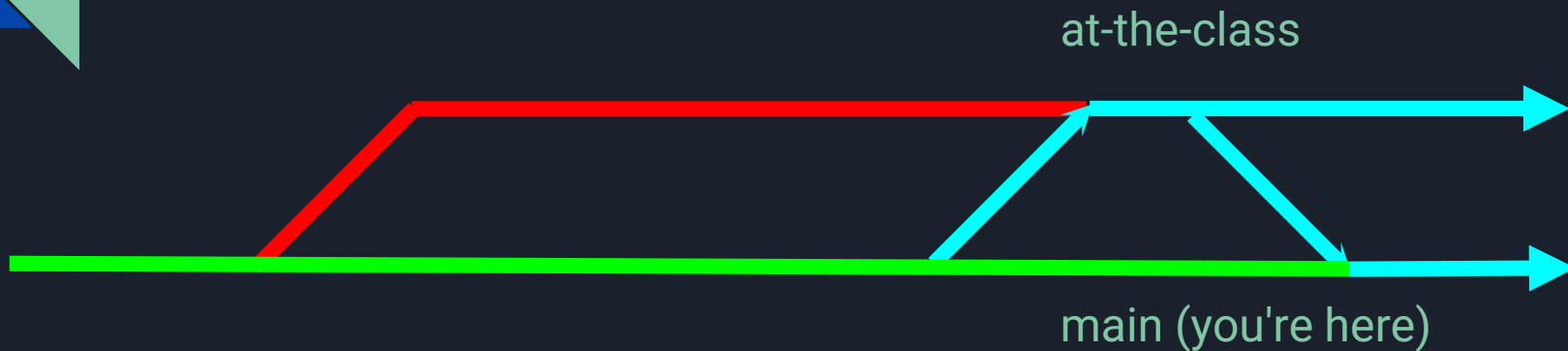
```
≡ story.txt
1 Adam wakes up in the morning.
2 He brushes his teeth.
3 He takes a bath.
4 He has breakfast.
5 Accept Current Change | Accept Incoming Change | Accept Merge
6 <<<<<< HEAD (Current Change)
7 Then he watched YouTube.
8 He goes to class.
9 =====
10 Then he checked Instagram.
11 He goes to class.
12 The lecturer was very boring.
13 He was confused with git.
14 >>>>>> at-the-class (Incoming Change)
```

Edit it to:

```
≡ story.txt
1 Adam wakes up in the morning.
2 He brushes his teeth.
3 He takes a bath.
4 He has breakfast.
5 Then he checked Instagram.
6 He goes to class.
7 The lecturer was very boring.
8 He was confused with git.
9
```

- Notice how Visual Studio Code understands git (if you open by folder) and highlights the <<<< HEAD line in green and >>>> at-the-class line in blue, with some helpful shortcuts.
- Just clear the <<<, >>> and === lines and edit the text file so you get the 'correct' version.
- I'm going to go with Instagram. You can choose to keep YouTube if you want. **Save. Git add, then commit.**

Branch Check



- Right now you have two branches. Since you've merged everything from `at-the-class` back into `main`, the two branches should be more or less the same. (If you chose YouTube instead of Instagram, the next time you merge `main` into `at-the-class`, the conflict will auto-resolve since git now knows how.)



Delete branch – `git branch`

1. Now we no longer need the `at-the-class` branch.
2. Run `git branch -d at-the-class` to delete it.

Note: You cannot delete branches with unmerged changes accidentally with this command. You have to use `-D` instead, but be careful – make sure there's nothing left in there you might want to keep.



Important Commands/Flags

git checkout **-b**

-b creates the branch

git branch **--delete** **thisBranch**

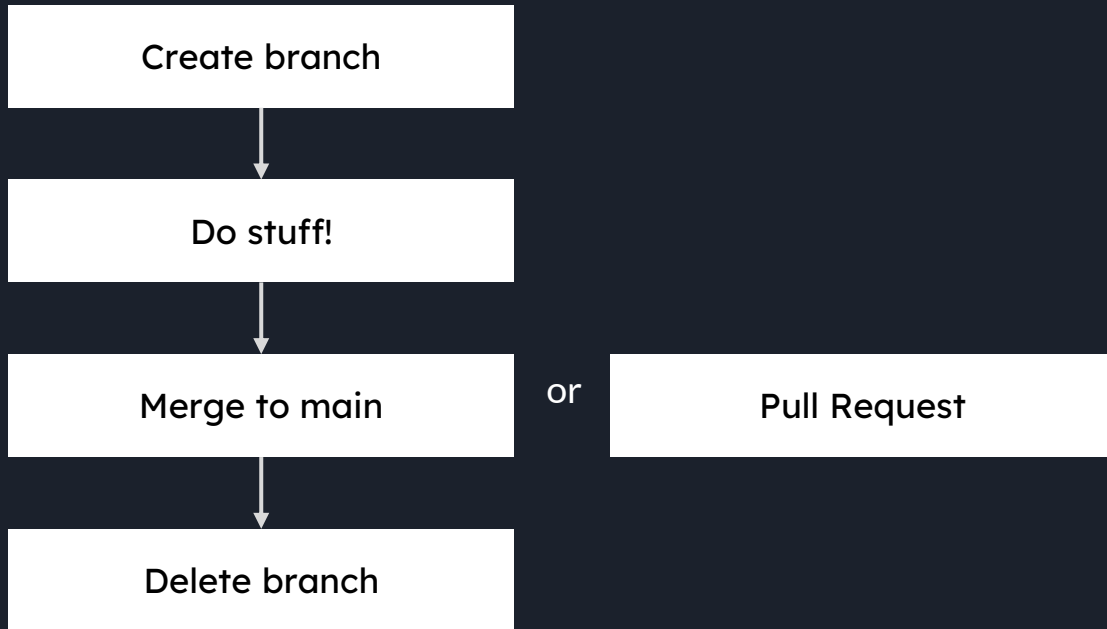
--delete will delete the branch called **thisBranch**

git push origin **--delete** **thisBranch**

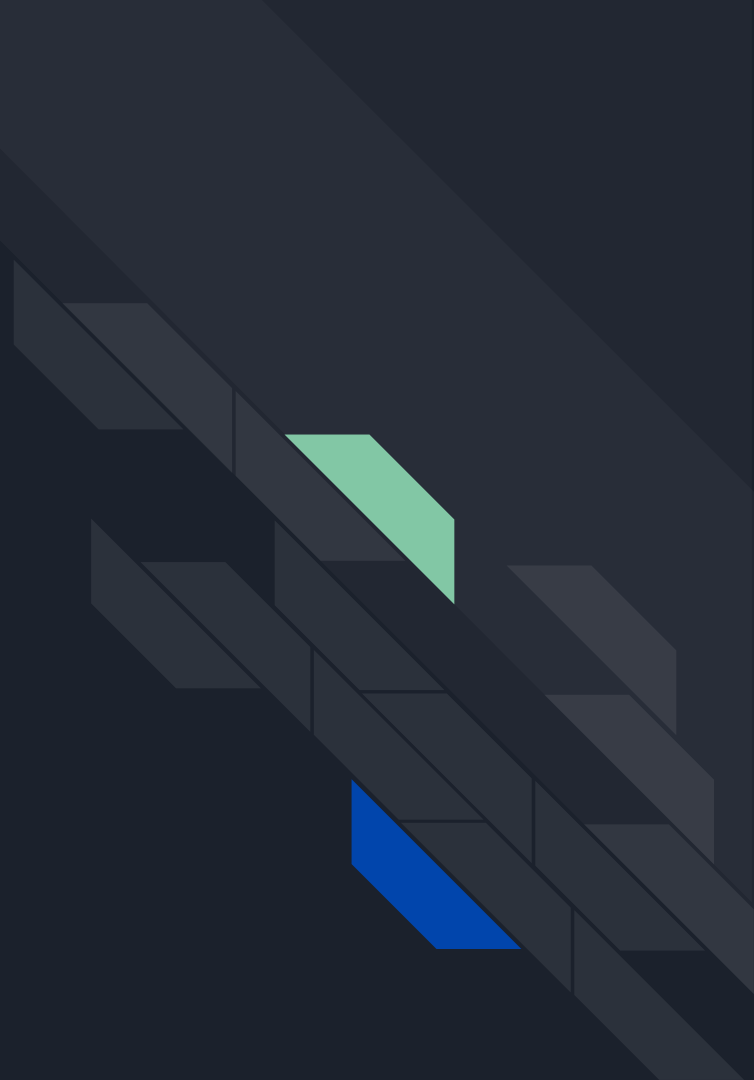
This tells Git to delete the remote branch. (on GitHub)



Branch Lifecycle



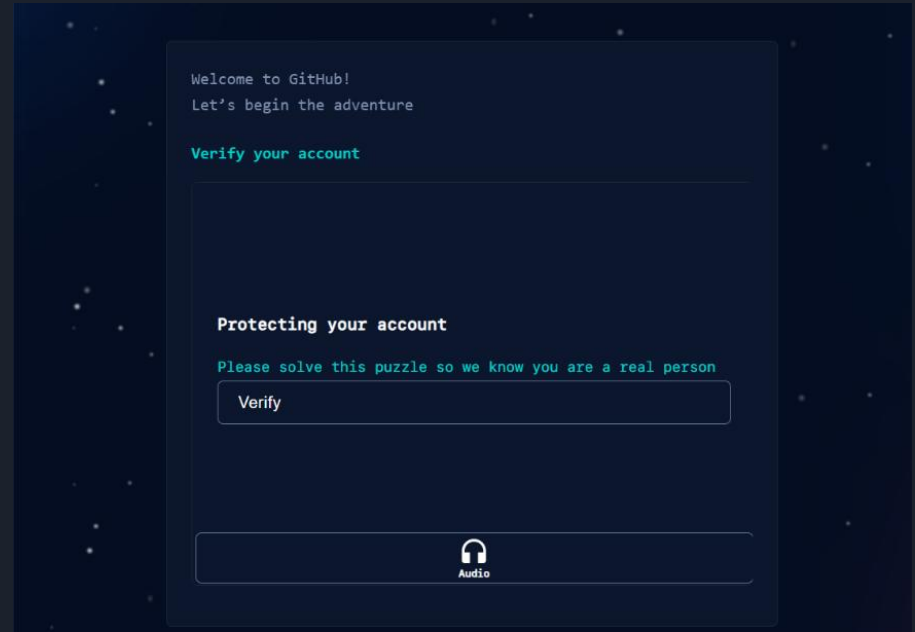
3. Using GitHub (Team Mode)



All Members – Setup GitHub

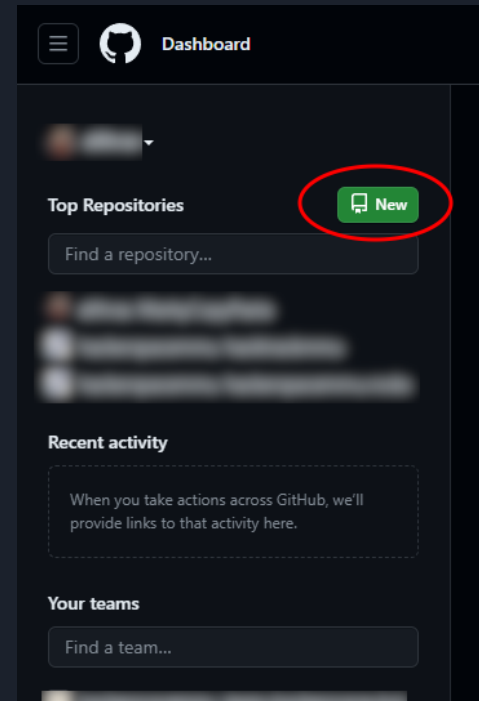
1. Go to <https://github.com/>
2. Hit Sign Up and fill in your details.
3. Verify using the verification code sent to your email.
4. Select "Just me" and "Student"
5. Check all the tools.
6. Select the "Free" option option and proceed. (Student stuff can be applied later.)

GitHub hosts the repo that all your team members will contribute to. You will 'push' to this repo.



Team Leader – Create the remote repository

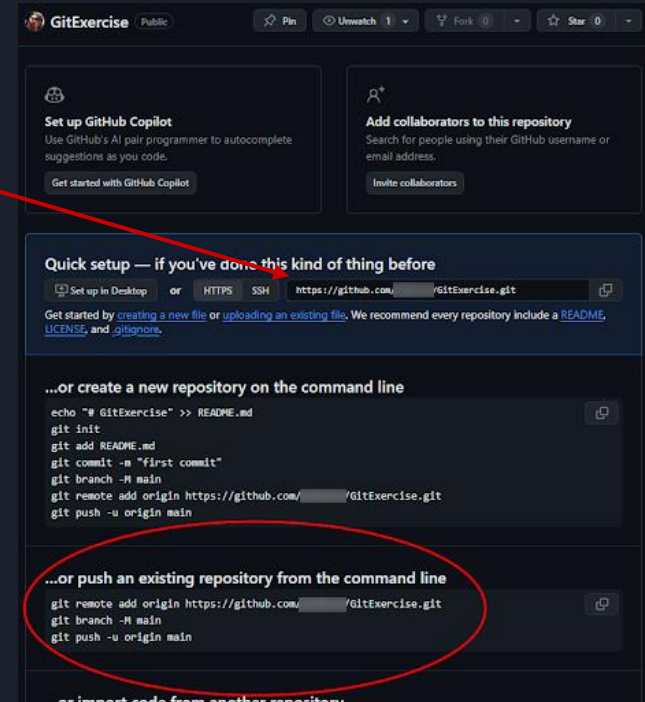
1. The repository on your computer is known as the 'local' repository. Now we need to make a 'remote' repository.
2. You **only need one** for your whole team, so choose a team leader to do this step. (All of you still need a GitHub account though!)
3. Click the "New" button. All you have to do on the next page is to give it a name. For this exercise, name it **GitExercise-GroupName**. **DO NOT** add a Readme File.



Team Leader – Push Local to Remote Repo

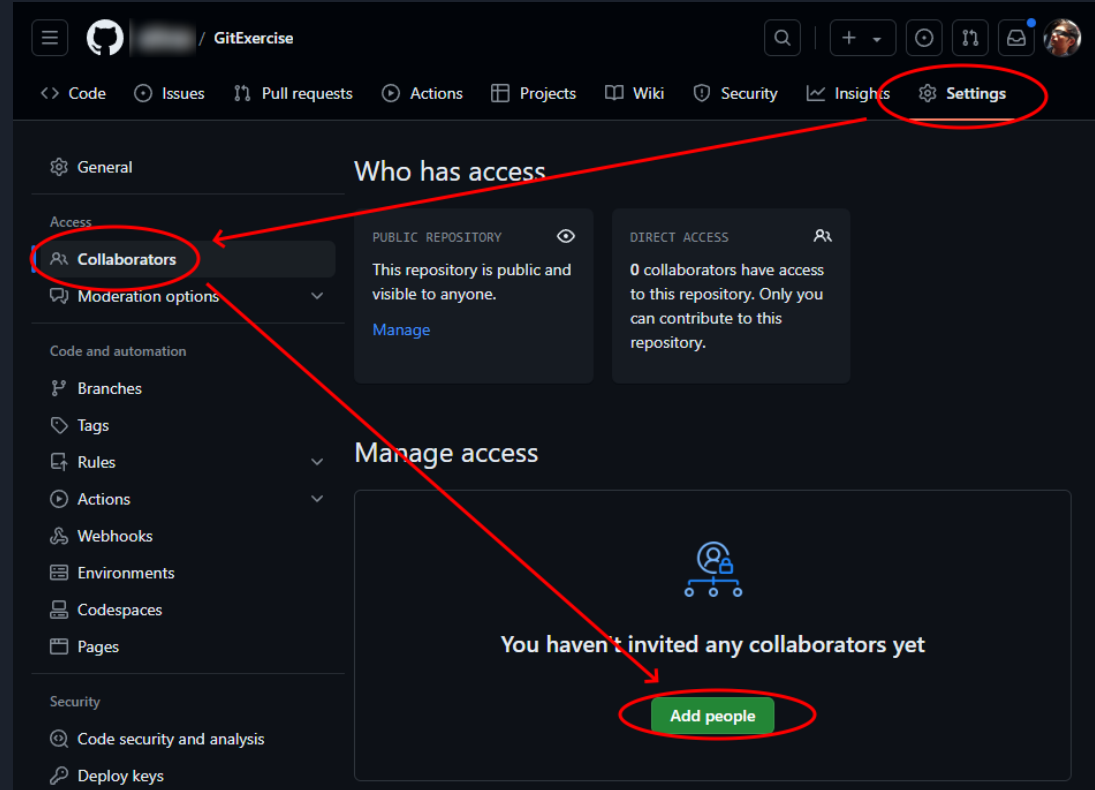
1. Still team leader only. On the next screen, you will get some instructions.
2. Go back to your command line, and run the `git remote add origin urlfromgithub` command given to you.
3. Then run `git push -u origin main`
4. You will be asked to sign into GitHub. Choose sign in with browser and authorise.

Notes: use `git remote -v` to see the remotes you set in case you put the wrong URL. Use `git remote set-url origin urlfromgithub` if you accidentally added the wrong URL the first time.



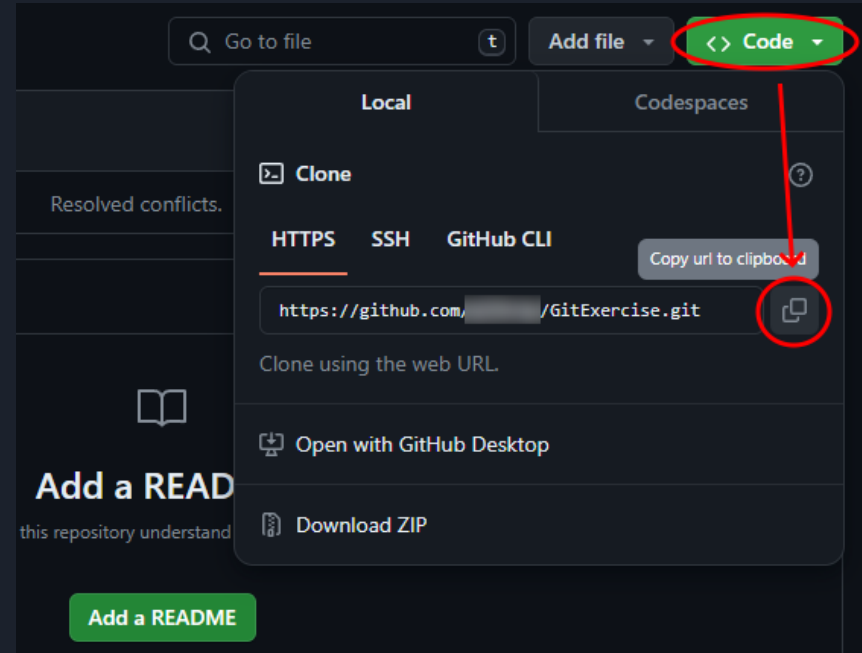
Team Leader – Add Your Members

1. Team leader only. In GitHub, add your team members! (See right screenshot)



Team Members - Clone the Repository

1. **Team members only.** Accept the invitation to the repository (check your email).
2. In the repository, copy the repository URL.
3. Then in your Projects folder (not GitExercise folder - this command will create a new folder for your project) run the command `git clone urlfromgithub` replacing urlfromgithub with the URL you copied.
4. After you clone, use `cd` command to enter the cloned project folder.
5. After this step, all team members will have local repos pointed at the same remote repo as 'origin'.
6. You're ready to branching, adding, committing and pushing! Try creating your own branch, making changes, committing and pushing your branch to GitHub (see next slide).



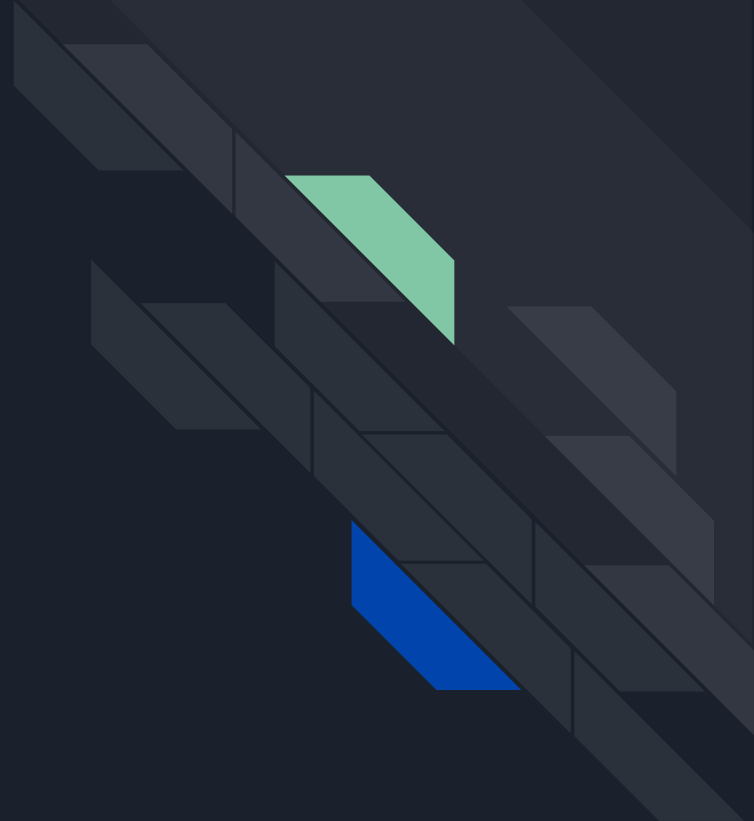


Working as a Team

1. Use branches when you're working. If not you will overwrite each other in the `main` branch.
2. You can push your local branches to the repository. Make sure you're IN your branch, then use `git push -u origin branchname`
3. When you want to merge, never merge your branch to your own local `main` branch (this will clash with the origin later). Use pull requests (next slide).
4. To get the branches your friends pushed to GitHub locally in your machine:
`git fetch origin branchname`
`git checkout branchname`

1. To make sure your codebase is up to date with everyone else's, regularly merge changes in the remote main into your local repository:
2. `git checkout main`
3. `git pull origin main`  **new!**
If main has no updates, then skip next two steps
4. `git checkout branchname`
5. `git merge main`

Alternative method to create a repository on GitHub





Using **git clone** to create repository on GitHub (As Leader)

With this method, you won't have to go back and forth between your Code Editor/Command line and GitHub. All you have to do is create a repository on GitHub, then use **git clone** to create a local copy of the repository on your computer.

1. Create a new repository on GitHub
2. Select "Initialise this repository with README file" (we do this to have a main branch upon repository initialisation, but a README is useful as a user guide for your project.)
3. Copy the git clone link
4. Use terminal to clone repository locally
5. Start working!

** Remember the git clone creates a folder for your repository, so you don't need to create a new folder to clone your project in.

Instead of running git clone in **C://Projects/GitTesting**, you can just run git clone in **C://Projects**, and git will create a new folder called GitTesting for you.

Here's what it would look like:

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Required fields are marked with an asterisk ().*

Owner *

Repository name *

JaxChoong

/

GitTesting

✔ GitTesting is available.

Great repository names are short and memorable. Need inspiration? How about [urban-doodle](#) ?

Description (optional)

☒ Public

☐ Private

Anyone on the internet can see this repository. You choose who can commit.

You choose who can see and commit to this repository.

Initialize this repository with:

☒ Add a README file

This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore

.gitignore template:

None

Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

Choose a license

License:

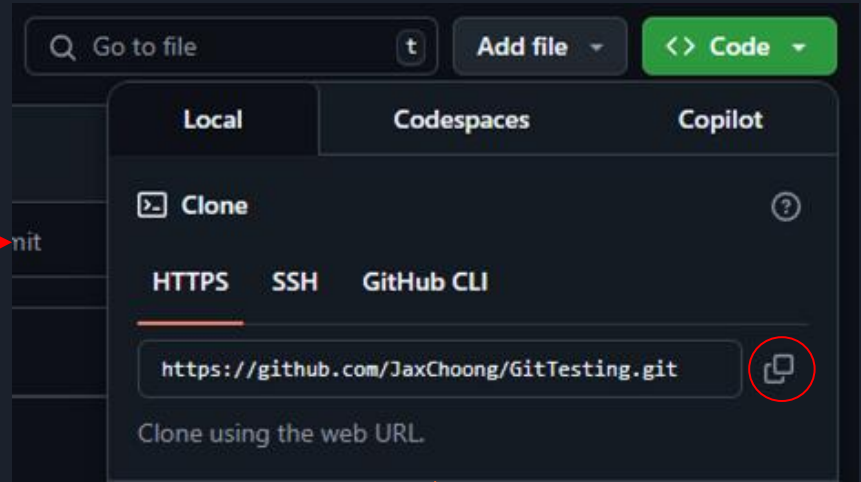
None

A license tells others what they can and can't do with your code. [Learn more about licenses.](#)

This will set `main` as the default branch. Change the default name in your [settings](#).

ℹ You are creating a public repository in your personal account.

Create repository

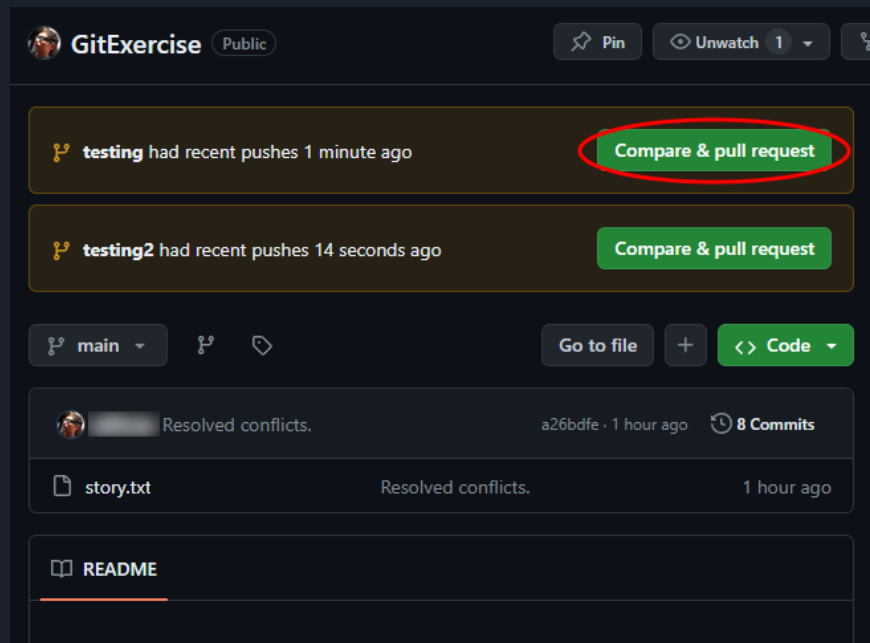


```
C:\Users\User>cd "VS Code Stuff"
C:\Users\User\VS Code Stuff>git clone https://github.com/JaxChoong/GitTesting.git
```

Pull Requests

1. Once you have pushed some branches onto your origin, go to your GitHub repository page. You will see a button to make pull requests. A pull request is a feature of GitHub, not git. (It's actually a request to merge.)
2. Click the button on GitHub. Set a title and describe why you're requesting the merge, then click "Create pull request."

Note: A good practice is to never create pull-requests that can't automatically merge. Checkout to main, pull it from origin (GitHub), merge it into your branch and resolve any conflicts, push back your branch back to GitHub, then only create the pull-request.



Open a pull request

Create a new pull request by comparing changes across two branches. If you need t

base: main < compare: testing2 ✗ Can't automatically merge. Do

Creating Pull Requests

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff comparisons here](#).

base: main

compare: newBranch

✓ Able to merge. These branches can be automatically merged.

 Add a title

Added explosion line

Add a description

Write

Preview



features added:
1. "He exploded." at line 10, at the bottom of the file.

Markdown is supported

Paste, drop, or click to add files

Reviewers

No reviews

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Use [closing keywords](#) in the description to automatically close issues

Helpful resources

Create pull request

Details to add:

- Title
- Description
- Reviewers (recommended)

Resolving Pull Requests

1. A good practice is to never approve your own pull requests - have another team member review it to make sure the code works without errors before it is merged into main. Once it's in main, everyone else gets it in their local repo.
2. Remember that the pull request screen have social features (comments) you can use to discuss the pull request.

<> Code Issues **Pull requests 1** Actions Projects Wiki Sec

Testing2 #2

Open wants to merge 2 commits into `main` from `testing2`

Conversation 0 Commits 2 Checks 0 Files changed 1

commented now Owner ...

No description provided.

added 2 commits 14 minutes ago

- Mastery! 199c8b4
- Resolved conflicts. 1f1b0c8

Add more commits by pushing to the `testing2` branch on `/GitExercise`

- Require approval from specific reviewers before merging
Rulesets ensure specific people approve pull requests before they're merged. Add rule ×
- Continuous integration has not been set up
GitHub Actions and several other apps can be used to automatically catch bugs and enforce style.
- This branch has no conflicts with the base branch**
Merging can be performed automatically.

Merge pull request ▾

You can also [open this in GitHub Desktop](#) or view [command line instructions](#).

Merge/pull/push cycle – one-way street:

PULL REQUEST on GitHub from your
remote mybranch to remote main
(once code in your branch is ready / does not
cause errors)

remote
mybranch

remote main

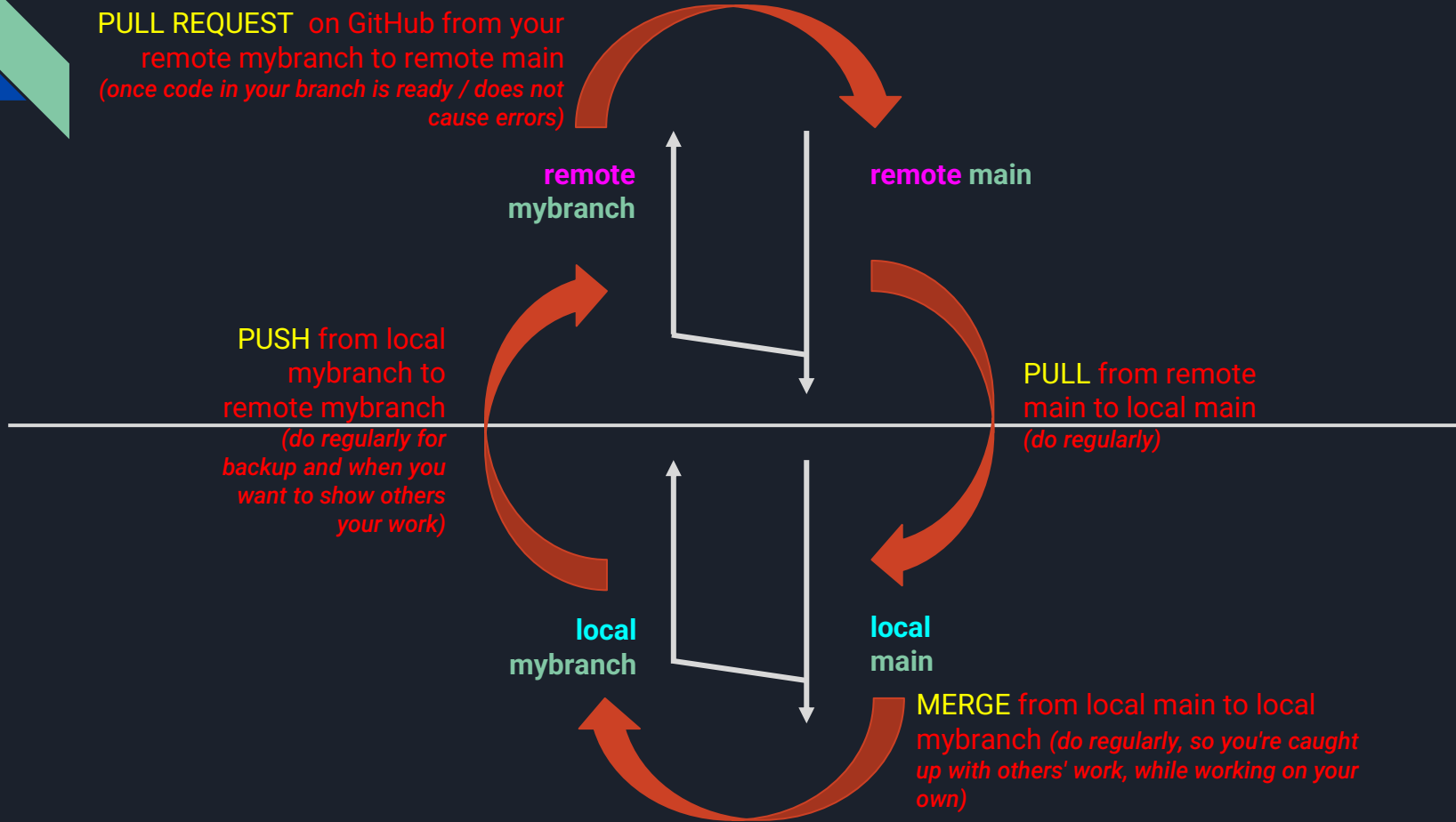
PUSH from local
mybranch to
remote mybranch
(do regularly for
backup and when you
want to show others
your work)

PULL from remote
main to local main
(do regularly)

local
mybranch

local
main

MERGE from local main to local
mybranch (do regularly, so you're caught
up with others' work, while working on your
own)



REMEMBER:

Commit often.
Merge often.

The background features a series of dark gray, three-dimensional rectangular blocks arranged in a perspective view, receding towards the right. A bright green triangle is positioned on the right side, partially overlapping the blocks. Below it, a blue triangle is also visible, also overlapping the blocks. The overall aesthetic is modern and technical.

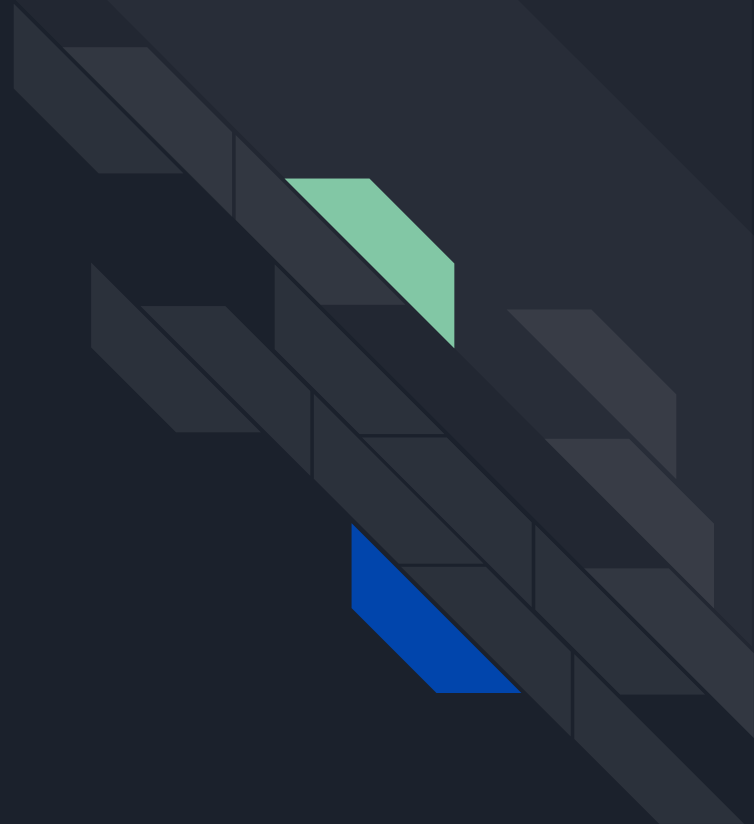


Now Go Finish the Story

- For the rest of the class, go write the story! Figure out who's supposed to write which parts, and practice branching, committing, pushing and pull-requesting your lines.
- Write in short lines to simulate coding.
- Use indentation (to simulate coding).
- Leave some empty space here and there to space the story out.

Remember - MERGE through pull requests on GitHub often. Do not wait weeks before attempting to merge - your codebases will become very different from each other, especially early on when your code is very short. Merge as often as you can.

4. Issues

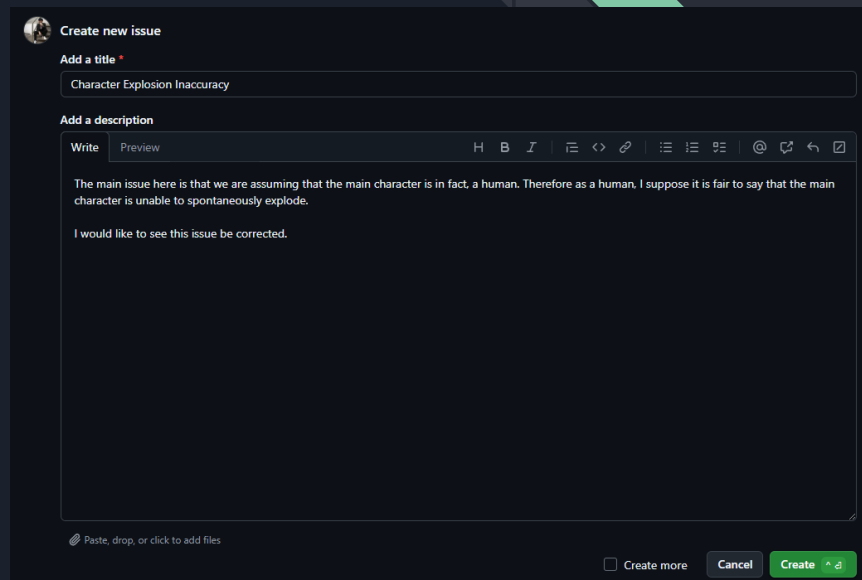
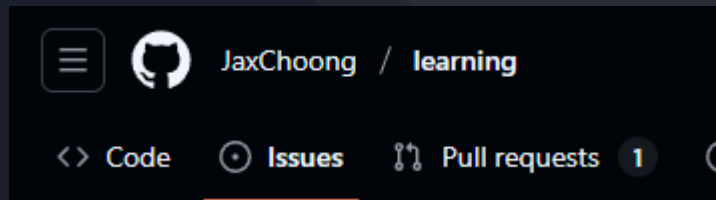


Creating an Issue

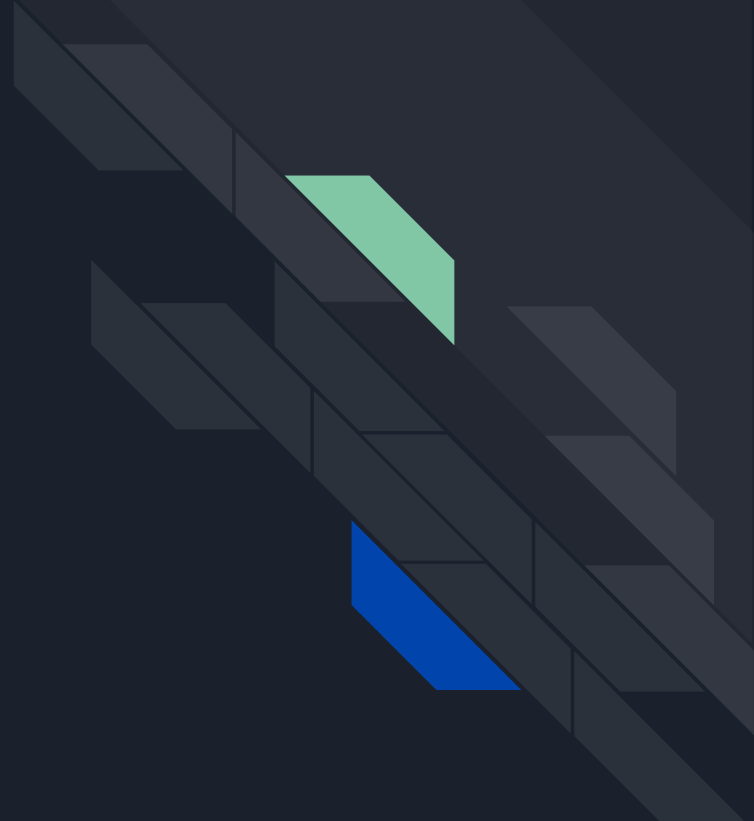
1. Navigate to “Issues” tab in GitHub Repository
2. Click on the green “New Issue” Button
3. Describe and create the Issue.

Important Notes:

- Include screenshots/code snippets for context
- Explain the issue



5. .gitignore and .env



.gitignore

A **.gitignore** file specifies intentionally untracked files that Git should ignore. These files are typically system-specific, auto-generated, or contain sensitive data that should not be versioned.

- ❖ Usually, this contains the “**.env**” directive to tell Git to omit/ignore the `.env` file when committing. This ensures that no sensitive information is open to the public.

For Example:

Sensitive Information(.env)

1. API keys
2. Login credentials
3. IP Addresses

Useless/spam files

1. Libraries (node_modules)
2. IDE/Editor settings (.vscode)
3. Caches (pycache)

Example .gitignore file

 .gitignore ●

```
1  # OS files
2  .DS_Store
3  Thumbs.db
4
5  # Logs
6  *.log
7  npm-debug.log*
8
9  # Dependency directories
10 node_modules/
11 venv/
12
13 # Build files
14 dist/
15 build/
16
17 # Secrets
18 .env
19
20 # IDE settings
21 .vscode/
22 .idea/
```

.env

A **.env** file is used to store environment variables or **sensitive information** for applications, particularly useful in development. These variables typically configure the app, set credentials, or specify API keys.

Common uses:

1. Configuration

- PORT = 3000
- DATABASE_URL=mysql://user:pass@localhost/db

2. API Keys

- API_KEY = 1234ABCD
- SECRET_KEY = ABCD1234

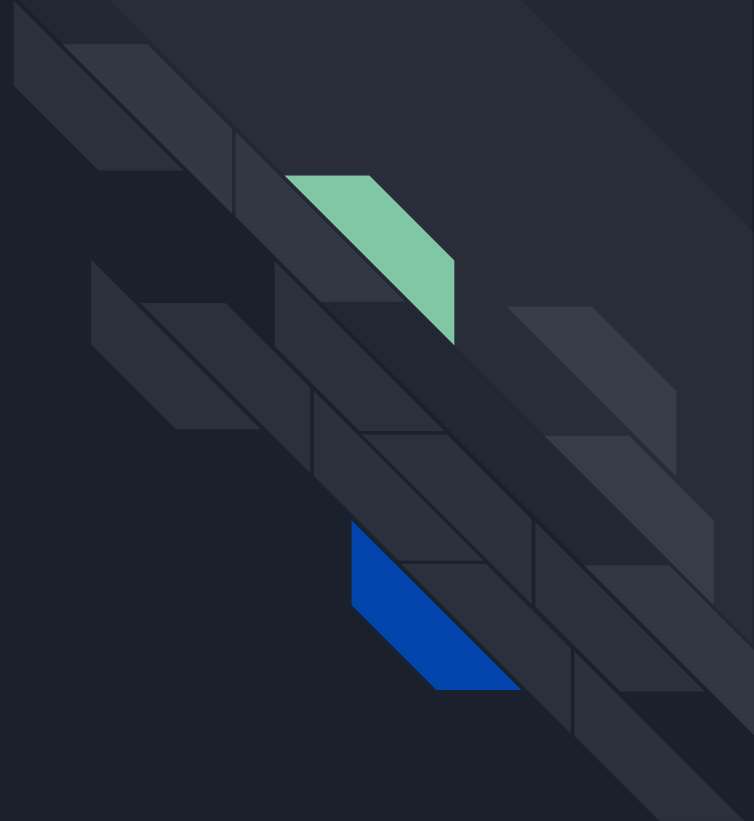
Example .env file

```
.env
1  # Server Configuration
2  PORT=3000
3  HOST=localhost
4
5  # Database Configuration
6  DB_HOST=127.0.0.1
7  DB_PORT=5432
8  DB_USER=username
9  DB_PASSWORD=password
10 DB_NAME=database_name
11
12 # API Keys
13 API_KEY=your_api_key_here
14 SECRET_KEY=your_secret_key_here
15
16 # Authentication
17 JWT_SECRET=your_jwt_secret_here
18 SESSION_SECRET=your_session_secret_here
19
20 # Miscellaneous
21 DEBUG=true
22 LOG_LEVEL=info
```

DO NOT COMMIT .ENV FILES

If you have are using a `.env` file, you **MUST** include it in your `.gitignore`, so Git does not commit it into GitHub and expose your sensitive information.

6. Quick Reference/Cheat Sheet

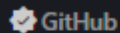


Useful VS Code Extensions



GitHub Copilot

Your AI pair programmer

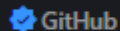


🕒 441ms



GitHub Copilot Chat

AI chat features powered by Copilot

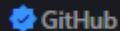


🕒 836ms



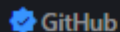
GitHub Repositories

Remotely browse and edit any GitHub repository



GitHub Pull Requests

Pull Request and Issue Provider for GitHub



🕒 163ms





GitHub Student Developer Pack

Benefits:

- GitHub Copilot
- Free website domain
- Notion Education Plan
- JetBrains subscription

And many more!

Sign up at <https://education.github.com/pack> with MMU Student ID Card.



Quick References (Local Only)

Starting (and going into) a new branch:

- `git checkout -b newbranchname`

Going into an existing branch:

- `git checkout branchname`

Committing code changes:

- `git add .`
- `git commit -m "Your commit description."`

Merging another branch into current branch:

- `git merge otherbranchname`

Deleting a branch once you're done with it:

- `git branch -d branchname`

Others:

- Seeing status `git status`
- See all branches `git branch`
- See last commit `git show`
- See commits `git log` (to quit press `q`)



Quick References (Remote)

Pulling from GitHub (origin)

- `git pull`

Daily routine to update branch with origin main

- `git checkout main`
- `git pull`
- `git checkout branchname`
- `git merge main`

Pushing change to others from inside a branch

- Make sure you're in the branch
- `git push -u origin branchname`

Pulling one branch from origin

- `git fetch origin branchname`
- `git checkout branchname`

Making a local copy of a GitHub repo

- `git clone repository-url`
(Command creates folder. Get URL from GitHub page)



Quick References (Additional)

Pulling everything from origin ([it's complicated](#)) – don't do this for big public open source projects!

- `git fetch --all` (get info of all branches)
- `git pull --all` (pull all the branches)
- `git checkout branchname`

Note: There is no rule that your local branch name must have the same name as a remote branch. You can pull a remote branch and track it using a local branch with a different name:

```
git checkout -b local_branch_name  
origin/remote_branch_name
```

Get rid of changes you haven't staged (never did git add) that you don't want to commit

- `git restore .`

Note: Many special things you want to do, you may want to Google clearly before running the command to see if that is what you want. Some command versions delete unstaged files. Some command versions only change tracked files.

Restoring back to a previous commit. The answer [is complicated](#). Read carefully.

- `git reset --hard xxxxxx` or
- `git revert xxxxxx`



Git through GUI

- You can use Git from within Visual Studio Code itself. Even without an extension, git recognises line additions, deletions and changes when it sees a Git repository.
- Install an extension for more commands. GitLens extension is very popular.
- Press Ctrl-Shift-P (or Cmd-Shift-P), type "Install Extensions", then search for GitLens. Learn to use it yourself.
- You can install a GUI execute Git commands.
- SourceTree's app is free and very powerful/beautiful.
<https://www.sourcetreeapp.com/>
- GUIs sometimes cannot be as specific as command lines however, so if you need to do something specific, sometimes you can Google up specific commands to execute. (Like **git rebase** for example - what does that do?)



Git through Faster Command Line

Windows

1. Create a file with the name `gs.bat` in your project directory.
2. Inside the file put the lines:
`@ECHO OFF`
`git status`
3. Try running `gs` inside your project directory.
4. You should move this `gs.bat` file [somewhere else in your PC](#) as it doesn't belong to your project.
5. Create a new batch file for every command with the filename being your shortcut, and the command inside being the command u want to run. [More details here.](#)

Mac

1. Open Terminal
2. Run `code .zshrc`
(You should get a file called `.zshrc` open in Visual Studio Code)
3. Add `alias gs="git status"` in one line
4. Save the file
5. Close and re-open Terminal
6. Go to your git folder in Terminal, and type `gs`. See what happens.
7. [More reference for other shortcuts you can make.](#)



THANK YOU FOR YOUR TIME AND ATTENTION!

Feel free to scan the QR codes below to give us feedback, and answer our survey!

Workshop Feedback





THANK YOU FOR YOUR TIME AND ATTENTION!

Feel free to scan the QR codes below to join our committee for this year!

IT Society

